

# Fayda MCP Python Library

Implementation guide | Importable package, no UI | 6 October 2026

## Delivery model

Publish a Python package named `fayda-mcp`, imported as `fayda_mcp`. A developer installs it into their own application and supplies their own registered Fayda relying-party configuration. The package exposes MCP tools and reusable identity-verification services. You do not operate a central tenant service or store customer credentials.

## Architecture

Agent -> developer-hosted MCP server -> library verification service -> Fayda eSignet. The developer application handles the customer browser redirect and registered callback. Your library constructs authorization URLs, exchanges codes, verifies signed responses, and returns minimal results.

## What you build

| Library owns                             | Integrating developer owns                              |
|------------------------------------------|---------------------------------------------------------|
| Typed MCP tools and result schemas       | MCP hosting and agent authorization                     |
| OIDC / PKCE and client assertions        | Fayda registration, client ID, and signing key          |
| JWT validation and claim evaluation      | Browser redirect and registered callback route          |
| Storage interfaces and optional adapters | Neon / Redis accounts and connection settings           |
| Framework-neutral Python API             | Application UI, consent presentation, business workflow |

No admin portal, onboarding UI, verification page, or frontend belongs in the core package. Fayda still requires the person to authenticate and consent through its own browser flow. The developer application delivers the authorization link and receives the callback.

## FastAPI and FastMCP

Standalone FastMCP exposes tools. FastAPI is an optional integration extra for callback routes and ASGI mounting. The core verification API must work without FastAPI. An MCP server is a running process, not merely an installed module. The developer must run or mount the server.

# 1. Runtime flow and public API

1. Developer configures the library with approved provider URLs, Fayda client ID, registered redirect URI, signing-key provider, session store, and result repository.
2. Agent calls `start_verification` with purpose, permitted checks, opaque application user reference, and idempotency key. Host supplies the authenticated caller context.
3. Library creates a request, generates state, nonce, and PKCE S256, and returns the authorization URL, request ID, and expiry. Bind the request to the intended application user or session before sending the link.
4. Developer application gives the URL to the person. The person authenticates and approves claims through Fayda eSignet.
5. Fayda redirects the browser to the developer registered callback with code/state or an error. Developer callback calls `library.complete_verification`.
6. Library consumes state once, checks expected context, exchanges code with `private_key_jwt` and the original PKCE verifier, validates ID token and signed `UserInfo`, and evaluates approved claims.
7. Library stores a minimal result. Agent polls `get_verification_result`. Library enforces caller binding, purpose permissions, and result expiry.

## Proposed Python API, not existing installed functionality

```
config = FaydaConfig(
    client_id=..., redirect_uri=...,
    issuer=..., authorization_endpoint=...,
    token_endpoint=..., userinfo_endpoint=..., jwks_uri=...,
)
service = FaydaVerificationService(
    config=config, key_provider=key_provider,
    sessions=session_store, results=result_repository,
    policy=verification_policy,
)
started = await service.start_verification(
    context=caller_context, purpose="onboarding",
    checks=["identity_verified"],
    application_user_ref="opaque-ref", idempotency_key="...",
)
# Host sends started.authorization_url to the intended person.
# Host callback passes the returned state/code and browser binding:
result = await service.complete_verification(
    code=code, state=state, browser_binding=browser_binding,
)
# A server-side stored context resolves request owner from state.
```

Callback is a public browser route protected by OIDC state and application-session binding. It does not require the person to hold an MCP access token. Never use a callback query parameter as a trusted caller identity.

## 2. Package folders and MCP integration

```
fayda-mcp/
src/fayda_mcp/
  __init__.py          # stable public exports
  config.py            # validated provider configuration
  context.py           # caller and application bindings
  exceptions.py        # typed safe failures
  schemas.py           # requests and minimal results
  service.py           # verification lifecycle
  policy.py            # purpose / claim permissions
  claims.py            # deterministic predicates
  oidc/
    client.py          # bounded async HTTP requests
    authorization.py   # URL, state, nonce, PKCE
    assertions.py      # private_key_jwt signing
    tokens.py          # signed response validation
    jwks.py            # trusted key fetch and rotation
    mapping.py         # localized claim normalization
  mcp/
    factory.py         # explicit FastMCP tool registration
    tools.py           # thin typed service calls
  storage/
    protocols.py       # session/result/audit interfaces
    memory.py          # local demos and tests only
    redis.py           # optional shared session adapter
    postgres.py        # optional durable result adapter
  secrets/
    protocols.py       # signing-key provider interface
    file.py            # developer-supplied local key
  integrations/
    fastapi.py         # optional callback router factory
examples/
  stdio_server.py
  fastapi_server.py
  .env.example
tests/unit/
tests/integration/
tests/contract/
docs/
pyproject.toml
uv.lock
README.md
LICENSE
```

### Dependency and installation design

Core: Pydantic, httpx, a maintained JOSE/JWT implementation, cryptography, and standalone FastMCP. Optional extras: fastapi, redis, postgres. PostgreSQL extra uses async SQLAlchemy with psycopg. Proposed installation: `pip install "fayda-mcp[fastapi,redis,postgres]"`. Pin validated dependency ranges and test the supported Python versions.

### Explicit MCP tools

`start_verification`, `get_verification_status`, `get_verification_result`, and optionally `cancel_verification`. No tool accepts signing keys, OTPs, biometrics, arbitrary provider URLs, or caller identity supplied as an untrusted argument. Host authorization supplies trusted caller context.

## 3. Hosting, storage, and configuration

### Two deployment modes

Local stdio: developer runs a package server entry point as a subprocess. Credentials come from developer-controlled configuration, environment, or protected key files. Bind data to the configured application context. No bridge OAuth service is required for this local transport. A registered callback receiver is still necessary for the citizen browser flow.

Remote HTTP: developer mounts FastMCP in their server and provides OAuth resource authorization, audience/scopes, caller mapping, and HTTPS. The library accepts a host authorization/context adapter. Agent authorization remains separate from Fayda citizen authentication. Keep provider credentials out of agent tool arguments.

### Neon adapter

Neon is optional and belongs to the integrating developer. Store requests, minimal results, consent metadata, policy version, caller binding, expiry, and safe audit events. Provide an explicit schema installation/migration command. Never create or migrate tables automatically on import. Use a configurable table prefix or schema to avoid collisions.

### Redis adapter

Redis is optional shared session storage. Store random state mappings, nonce, PKCE verifier, request reference, caller/app binding, and expiry. Atomically consume state once. Redis outage requires a new verification flow. A storage interface lets developers replace Redis with an equivalent atomic database implementation.

Memory storage is for single-process demos and tests. Callback and MCP calls must share the same service/storage instance. Production with multiple processes needs shared storage. Keep the durable request record separate from temporary OIDC state.

### Example environment, loaded by host application

```
FAYDA_CLIENT_ID=business_registered_client_id
FAYDA_REDIRECT_URI=https://developer-app.example/auth/fayda/callback
FAYDA_ISSUER_URL=https://APPROVED_ISSUER
FAYDA_AUTHORIZATION_URL=https://APPROVED_AUTH_ENDPOINT
FAYDA_TOKEN_URL=https://APPROVED_TOKEN_ENDPOINT
FAYDA_USERINFO_URL=https://APPROVED_USERINFO_ENDPOINT
FAYDA_JWKS_URL=https://APPROVED_JWKS_ENDPOINT
FAYDA_SIGNING_KEY_PATH=/run/secrets/fayda_signing_key.json

# Optional adapters: host controls accounts and credentials.
DATABASE_URL=postgresql+psycopg://USER:PASS@HOST-pooler/DB?sslmode=require
DATABASE_MIGRATION_URL=postgresql+psycopg://USER:PASS@HOST/DB?sslmode=require
REDIS_URL=redis://default:PASSWORD@HOST:6379/0
FAYDA_SESSION_TTL_SECONDS=600
FAYDA_RESULT_TTL_SECONDS=900

# Remote HTTP hosting only, configured by the host.
MCP_RESOURCE_URL=https://developer-app.example/mcp
MCP_AUTH_ISSUER=https://developer-auth.example/
MCP_AUTH_JWKS_URL=https://developer-auth.example/jwks
```

Variable names are proposed. A key-provider object is preferable when the host already uses a vault or KMS. Do not load .env, open connections, fetch JWKS, or start servers automatically during import. Initialize explicitly and close clients through an async context manager.

## 4. Classified coder tasks and acceptance criteria

### A1 | Package foundation

Build: Create src layout, public exports, pyproject extras, settings, typed errors, and async lifecycle.

Acceptance: Import performs no network calls or migrations. Core installs without optional FastAPI, Redis, or database dependencies.

### A2 | Storage contracts

Build: Define atomic consume-state, request create/update, result read, audit append, expiry, and idempotency interfaces. Implement memory adapter.

Acceptance: Duplicate callback state fails. Concurrent completion does not finalize twice. Adapter tests apply to every backend.

### B1 | OIDC authorization

Build: Generate random state/nonce, PKCE S256, approved scopes/claims, request bindings, and an authorization URL.

Acceptance: Each request has unique short-lived state. Unsupported purpose/claim requests fail before redirect.

### B2 | Code exchange and JWT validation

Build: Sign private\_key\_jwt, exchange code, fetch approved UserInfo, validate signatures and authentication context.

Acceptance: Wrong issuer/audience, expired or unsigned JWT, incorrect nonce, mismatched sub, reused state, and bad PKCE all fail.

### B3 | Claims and results

Build: Normalize provider fields and implement permitted identity and age checks. Distinguish unavailable from false.

Acceptance: Missing DOB returns unavailable. No tokens, raw biometric data, or unnecessary demographics enter MCP results.

### C1 | MCP factory

Build: Build an explicit tool-registration factory with a host-provided caller/authorization adapter.

Acceptance: A real MCP client completes start/status/result flows. Caller A cannot read caller B results.

### C2 | FastAPI integration extra

Build: Provide optional callback router and an example combined FastAPI/FastMCP lifecycle. Require host session-binding hooks.

Acceptance: No UI templates are shipped. Redirect/callback works with the host registered URI. MCP lifespans run correctly.

## 4. Coder tasks: storage and release

### D1 | Redis and Neon adapters

Build: Implement TLS connections, TTL, atomic state consumption, tenant/app namespace, small pools, and explicit schema migrations.

Acceptance: Two application processes share a flow. Redis restart expires unfinished sessions safely. Neon retains completed results.

### D2 | Examples and documentation

Build: Provide stdio and remote HTTP examples, configuration, callback wiring, credential ownership, and cleanup guidance.

Acceptance: A developer configures their own Fayda client and runs the sandbox flow without editing library internals.

### E1 | Tests and release

Build: Add storage, security, MCP contract, sandbox integration, wheel/sdist, optional-extra, and supported-version checks.

Acceptance: Clean environments install the built package and run examples. No credentials ship in source distributions or wheels.

## 5. Security contract and release sequence

### Required validation

Use approved provider endpoints and trusted JWKS. Allow only configured signature algorithms. Validate issuer, audience, expiry, issued-at bounds, nonce in the ID token, azp where applicable, and matching ID-token/UserInfo subject. Refresh unknown signing keys with bounded requests. Never authorize using decoded-but-unverified payloads.

State links each callback to its original request and expected application context. A valid Fayda login alone does not prove the person owns an existing application account. The host must bind or confirm account linking. Browser binding and consent presentation belong to the application integration contract.

The library computes claim predicates through deterministic Python logic. A derived age check is not a Fayda-signed predicate or a zero-knowledge proof. Fayda authentication is not proof of citizenship and is not complete KYC compliance.

### Credential handling

Use the developer business credentials. Support key providers and rotation without putting private keys in database rows, prompts, tool inputs, exceptions, or logs. Never forward MCP access tokens to Fayda. Do not expose raw identity tokens as MCP results.

### Build sequence

1. Core configuration and storage interfaces. 2. OIDC start/callback and signature checks. 3. Claims and results. 4. MCP tool factory. 5. Optional FastAPI, Redis, and Neon integrations. 6. Sandbox tests, documentation, package build, and release.

### Release completion

Developer imports the package, configures their own Fayda client and callback, starts an MCP server, and gives a customer a verification link. Customer completes Fayda authentication. The library validates signed responses. The authorized agent retrieves a minimal result. No package-owned UI or central business-credential database is required.

### Primary implementation references

Fayda: client ID, signing key, Authorization Code and PKCE. Confirm deployed endpoints, allowed claims, assurance, key format, and relying-party approval through onboarding.

<https://nidp.atlassian.net/wiki/spaces/~712020e898f1e6ab3543aabe17e0865c3305f5/pages/845905921>

FastMCP integration and HTTP hosting: <https://gofastmcp.com/integrations/fastapi>

MCP transport authorization: <https://modelcontextprotocol.io/specification/latest/basic/authorization>

Neon pooling: <https://neon.com/docs/connect/connection-pooling>

SQLAlchemy PostgreSQL driver: <https://docs.sqlalchemy.org/en/20/dialects/postgresql.html>

Package API names, file structure, and settings in this guide are proposed interfaces to implement. Provider facts follow the documentation checked during this project on 6 October 2026.